

Data Structure

It means structure of data.

OR

In data structure data may be organised in many different way like logical, increasing, decreasing or alphabetical. It totally depend on the particular organisation.

Basic terminology of data structure

- Data: Data is a value or set of value. Data item refers to a single unit of a value. Data items may be divided into subitem. It is also called group item.

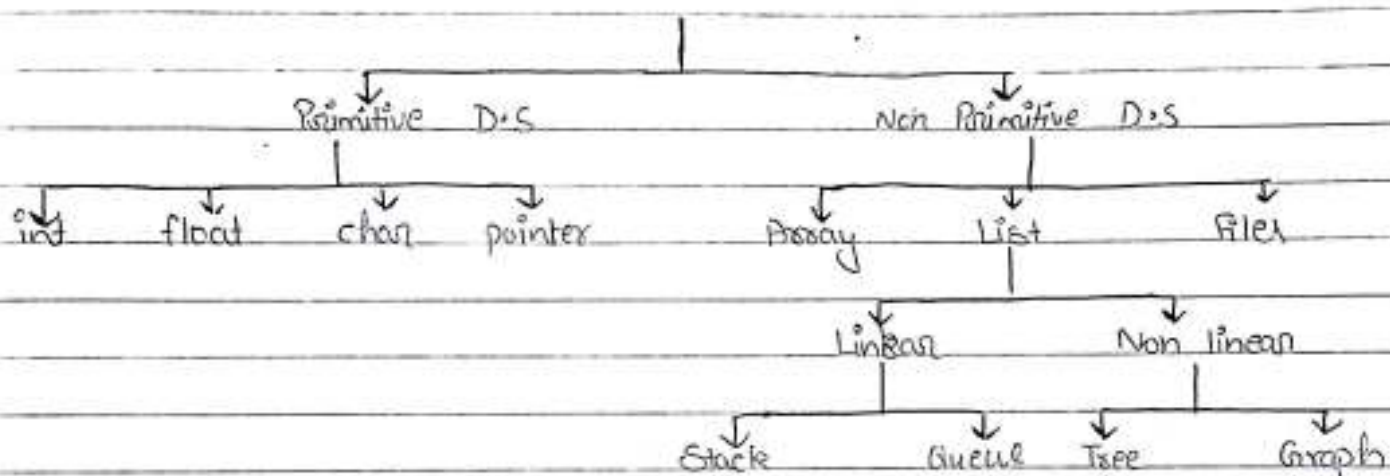
Name	Age	Gender	Roll no	Branch
X	20	M	01	BCA
Y	19	F	02	BCA
X	20	M	03	BCA
Z	21	F	04	BCA

- Entity: Entity is an object existing in a real world with logical or physical meaning.
- Attributes: It is the property of that particular entities.
- Primary Key: Uniquely identified by any entity is known as a primary key.

- Record: Collection of all attributes of a given entity is known as a primary records.
- Files: Collection of all records of an given entities is known as files.
- Entity Set: Where we collecting each and every collection mention in a table that table is known as entity set.

Classification of Data Structure

It is classified in two broad categories:



- Primitive Data Structure: This is basic data structure. Directly operated by machine instruct. They had the different representation on different computers.
Example: int, float etc.

- Non-Primitive Data Structure: It is derived from primitive D.S. This d.s emphasize on structuring of group of 'homogeneous or heterogeneous data type'.

Algorithm & Efficiency of Algorithm

An algorithm is well defined list of steps to solve any particular problem. Always written in own language.

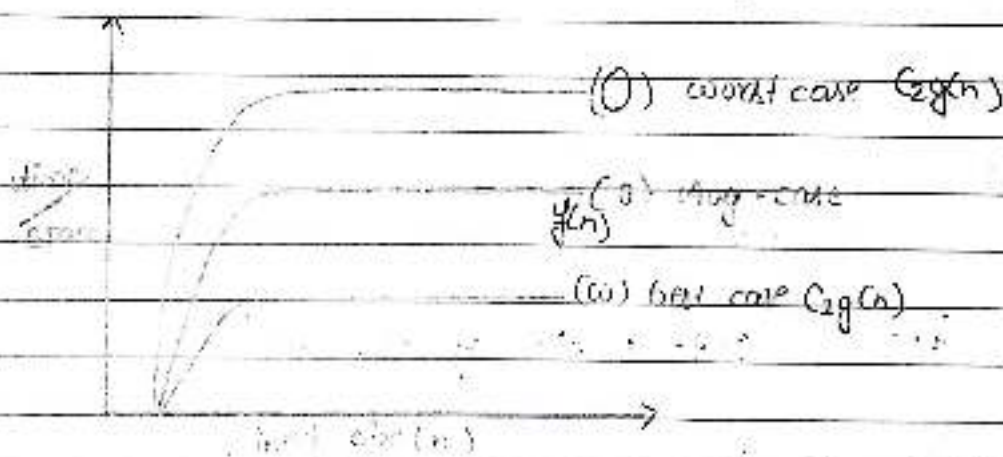
- * Efficiency of algorithm: It is too big measurement of efficiency of algorithm.

(1) Time

(2) Space

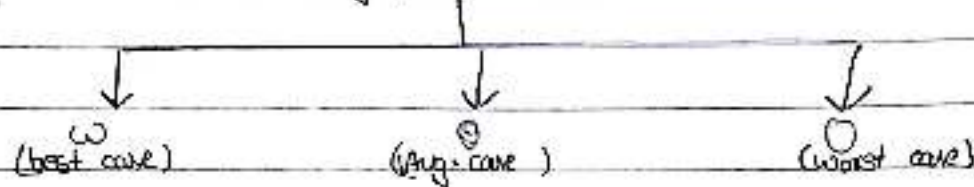
Complexity of an Algorithm

Complexity of an algorithm is a function $f(n)$ which gives the running time and or space in the term of input size n .



Asymptotic Notation To select the best algorithm we need to check the efficiency of each algorithm. Efficiency can be measured by computing the time complexity of each algorithm. It represents three notations for time complexity:

Asymptotic Notation



① Avg. case $(a) = C_1 g(n) \leq f(n) \leq C_2 g(n)$

② Best case $(w) = C_1 g(n) \leq f(n)$

③ Worst case $(O) = C_2 g(n) \geq f(n)$

Abstract Data Type (A.D.T.)

- * Model of data type
- * Properties of data type.
- * Set of operations performance on that data.

ADT	
Properties of data	
Operations	

Definition: It is a mathematical model with a collection of operations that can be performed on a particular data type.

$ADT = \text{Data} + \text{Set of operations}$

Example - ADT Integer $\Rightarrow$

Integer ADT	
Properties Number +ve / -ve Operations Addition Subtraction Multi. Division	

In static we fixed the size of the memory before compilation of program.

We fixed the size of memory on run time and that size may be change accor. to their need.

All the element are stored in a sequential manner.

All the element are stored in a random manner.

We need to stored the address of 1st block of memory that address is known as base address.

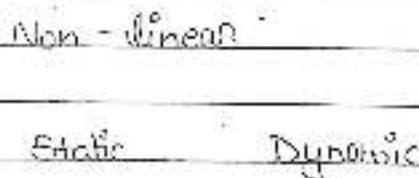
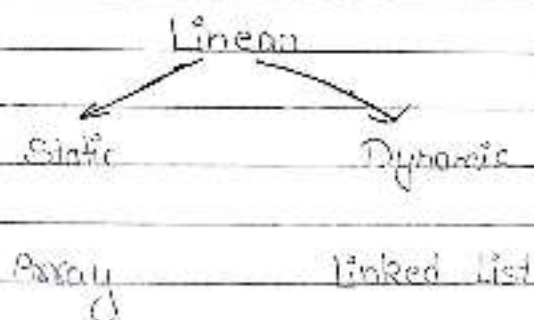
We need to stored the address of every variable.

Wastage of memory.

Nominal wastage of memory.

Static is more efficient than dynamic.

Dynamic is less efficient.



ARRAY

Data structure is either linear or non-linear. It is said to be linear if its elements form in sequence. Such linear list are stored in memory in two way:

1

- ① If such linear list are stored in static memory allocation that representation is known as Array.
- ② If elements are such linear list element of such linear list are represented in dynamic ~~st~~ memory allocation that representation is known as linked list.

OR

* Finite set of collection of homogeneous type of data element and all the data elements are stored in static memory allocation is known as a Array.

Length of Array or size of Array:

On the basis of given formula we can calculate the length of Array or size of Array =

$$\boxed{\text{Length of Array} = \text{UB} - \text{LB} + 1}$$

where, LB is Lower Bound, the smallest index of the Array.
UB means upper Bound is the longest index of the Array.

Ques

Disadvantage: Array in C has no bound checking

Note: No. of elements in an Array is equal to the length of size of the array

Representation of 1-D Array in memory

Let LA mean Linear Array be a memory of computer. We store the element of Array in memory. Computer doesn't need to store the address of every element of Array in memory.

But need to store the address of first element of Array in memory that address is known as base address.

Using base address we can calculate the address of every element in memory.

Formula

$$\text{Address}[A[K]] = B + w(K - LB)$$

Example :

5	11	18	7	4	3
---	----	----	---	---	---

LB $\swarrow$ 200 1 2 3 4 5



$$\begin{aligned} \therefore \text{Address}[A[4]] &= B + w(K - LB) \\ &= 200 + 2(4 - 0) \\ &= 200 + 8 \\ &= 208 \end{aligned}$$

* w based on data type.

Q.1 Consider the linear array $A(S:50)$, $B(-5:10)$, $C(18)$

(1) No. of element in each array

(2) Suppose $\text{Base}[A] = 300$ and $w = 4$ word/memory cell for A . Find the address of $A[15]$, $A[35]$, $A[55]$

Solution-2- (i) $\text{Address}[A[15]] = 300 + 4(15 - S)$
 $= 300 + 40 = 340$

(ii) $A[35] = 300 + 4(35 - S)$
 $= 300 + 4 \times 30 = 300 + 120$
 $= 420$

(iii) $A[55] = 300 + 4(55 - S)$
 $300 + 4 \times 50 = 300 + 200$

$A[55]$ doesn't exist in array $A = S:50$
So, we can't find address of $A[55]$.

Representation of 2-D Array in Memory

A 2-D Array is also known as 2-D matrix where two subscript variable are used $[I, J]$. 'I' represent the 'row' and 'J' represent 'column'.

A 2-D Array A is a collection of $(m \times n)$ element, where m represent row and n represent column.

Most programming language are used to store a 2-D array in row by row that representation is known as Row Major Order. Or Column by column that representation is known as Column Major Order.

	1	2	3	4	5
1	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)
2	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)
3	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)
4	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)

① Row Major Order =

$$\text{Address}[A[i, j]] = B + w[n(i - L_1) + (j - L_2)]$$

$$\text{Address}[A(i, j)] = B + w[m(j - L_2) + (i - L_1)]$$

Q. A 2-D Array defined as a $A[4 \dots 7, -1 \dots 3]$ requires 2 bytes of storage space for each element. If the array is stored in row and column major order the calculate the address of elements at location $A[6, 2]$. The B.A is 100.

∴ Row major order

$$\begin{aligned} A[6, 2] &= 100 + 2 [5(6-4) + (2+1)] \\ &= 100 + 2 [10 + 3] \\ &= 100 + 26 = 126 \end{aligned}$$

∴ Column major order

$$\begin{aligned} A[6, 2] &= 100 + 2 [4(2+1) + (6-4)] \\ &= 100 + 2 [12 + 2] \\ &= 100 + 28 = 128 \end{aligned}$$

Q.2 Each element of an array $X[30][50]$ required 4 bytes of storage. Base address of X is 2500. Determine the location of $X[10][10]$ when array

Row Major Order =

$$\begin{aligned} A[10, 10] &= 2500 + 4 [50(10-0) + (10-0)] \\ &= 2500 + 4 [500 + 10] \\ &= 2500 + 4 \times 510 \\ &= 2500 + 2040 = 4540 \end{aligned}$$

Representation of 3-D Array in Memory

It can be done in two ways:

(1) Row major Order

(2) Column Major Order

• Address $[I, J, K] = B + w[E_3 + L_3(E_2 + E_1 L_2)]$ — Row

• Address $[I, J, K] = B + w[E_1 + L_1(E_2 + E_3 L_2)]$ — Column

where, B = Base Address

w = word / memory cell

E_1, E_2 & E_3 are denoted by E_i i.e. known as Effective Index.

• Effective Index: E_i of L_i is the no. of preceding index in the total index set.

Formula $\Rightarrow E_i = K_i - L_i$

L_1, L_2 & L_3 are length of subscript value of 3-D Array

Q. $A(30, 40, 50)$, $B = 100$, $w = 4$ w/m.c.

Find the loc. of $A(20, 30, 40)$

$\therefore L_1 = 30, L_2 = 40, L_3 = 50$

$E_1 = 20, E_2 = 30, E_3 = 40$

• Row = $A(20, 30, 40) = 100 +$

Q.1 Consider a 3-D Array defined as $x[0-9, 0-49, 0-29]$.

Assume that storage for array start at 2000 in memory and 4 byte space are required to hold each element of an array. compute the actual address of the array $x[5, 20, 10]$

Assuming that the array stored in row major and column major order

$$\therefore L_1 = 10, L_2 = 50, L_3 = 30$$

$$K_1, K_2, K_3 = 5, 20, 10 \quad \text{and} \quad E_1 = 5, E_2 = 20, E_3 = 10$$

$$\begin{aligned} \therefore \text{Column Major Order} &= 2000 + 4[5 + 10(20 + 10 \times 50)] \\ &= 2000 + 4[5 + 10 \times 520] \\ &= 2000 + 4[5205] \\ &= 2000 + 20820 = 22820 \end{aligned}$$

Row Major Order:

$$\begin{aligned} &= 2000 + 4[10 + 30(20 + 5 \times 50)] \\ &= 2000 + 4[10 + 30 \times 230] \\ &= 2000 + 4[10 + 6900] \\ &= 2000 + 4[6910] \\ &= 2000 + 27640 \\ &= 29640 \end{aligned}$$

B. Consider a multidimensional array $A[90, 30, 40]$ with $B = 1000$.

Calculate the address of $[10, 20, 30]$ in row major order. Assume that the first element is stored at $A[2, 2, 2]$ and each element take 2 byte of space.

$$L_1 = 90, L_2 = 30, L_3 = 40 \quad \text{and} \quad K_1 = 10, K_2 = 20, K_3 = 30$$

$$\therefore E_1 = 2, E_2 = 2, E_3 = 2$$

$$\begin{aligned}
 \therefore \text{Row Major Order} &= 1000 + 2[28 + 39(18 + 3 \times 29)] \\
 &= 1000 + 2[28 + 39(18 + 232)] \\
 &= 1000 + 2[28 + 39 \times 250] \\
 &= 1000 + 2[28 + 9750] \\
 &= 1000 + 2 \times 9778 \\
 &= 1000 + 19556 = 20556
 \end{aligned}$$

Sparse Matrices

Relatively a high proportion of 0 entries in compare to non-zero element. In this matrix no. of row and column are equal such type of matrix is known as a 'Sparse Matrix'. This is also known as N^2 Matrix.

	$C(n)$			
	0	1	0	3
$R(n)$	7	0	0	9
	0	3	0	2
	0	0	0	1

Classification of Sparse Matrix

It is classified is based on entry of the non-zero element. They are classified in two categories:

- (1) Triangular Sparse Matrix
- (2) Toldiagonal Sparse Matrix

• Triangular S.M : It is divide in two parts :

(a) Lower Triangular Sparse Matrix

(b) Upper Triangular Sparse matrix

* In lower triangular S.M entry of the non-zero element to the lower part of matrix and upper part treated as empty.
The empty part is replaced by 0.

* In upper triangular sparse matrix entry of the non-zero element to the upper part of matrix and lower part treated as empty
and the empty part is replaced by 0.

Ex -

1		
2	0	
3	5	2

1	0	0
	5	2
		3

• Tri-diagonal S.M : In tri-diagonal S.M entry of the non-zero element in the main diagonal of matrix

	1		
		2	
			3

Application of Sparse Matrix

(1) Dense Matrix :

(2) Digital Image Processing

STACK

Stack is a linear type of data structure. It work on 'LIFO' principle.

LIFO stand for "Last In First Out".

* These are two basic operation performs in stack:

- (1) Push
- (2) Pop

- Push means 'inserting the element' into stack.
- Pop means 'deleting the element' from the stack.

Data may be pushed or popped at only one end of the stack. That end is known as 'Top'.

Note: Some conditions of stack:

- If pushing the element into the stack

$$TOP = TOP + 1$$

- If popping the element from the stack, $TOP = TOP - 1$

• If $TOP \geq MAX - 1$ that means overflow condition occur.

- If $TOP = -1$ that means underflow condition occur in stack.

Write an algorithm to Push the data element into 'STACK'.

- (1) Create a 'STACK'.
- (2) Check the overflow condition. If $(TOP == MAX - 1)$ then, print overflow condition occur.
- (3) Exit
- (4) Push data element into STACK.
 - (a) Set $TOP = TOP + 1$.
 - (b) Set $STACK[TOP] = Element$
- (5) Exit.

* Write a program function in C to push data element into stack

```
void Push()  
{  
    int element;  
    if (TOP == MAX - 1)  
    {  
        printf("Stack is overflow");  
    }  
    else  
    {  
        printf("Enter the element");  
        scanf("%d", &element);  
        TOP = TOP + 1;  
        STACK[TOP] = element;  
    }  
}
```

Write an algorithm to POP the element from the STACK.

- (1) Create an STACK 'S'.
- (2) Check the underflow condition. If (TOP == -1) then print underflow condition occur.
- (3) Exit.
- (4) POP the element from the STACK.
 - (a) Set the element = STACK[TOP]
 - (b) Set TOP = TOP - 1;
- (5) Exit.

BCK: /* Write a function in C to POP data element into STACK.

```
void POP ()  
{  
    int element;  
    if (TOP == -1)  
    {  
        printf("Stack is underflow");  
    }  
    else  
    {  
        int element = STACK[TOP];  
        TOP = TOP - 1;  
        printf("%d", has been deleted, element);  
    }  
}
```



```
#include <stdio.h>
#include <conio.h>
#define MAX [5];
int STACK [MAX];
int TOP = -1;

void PUSH ( )
{
    int element ;
    if (TOP == MAX-1)
    {
        printf("STACK is full");
    }
    else
    {
        printf("Enter the element");
        scanf("%d", &element);
        TOP = TOP + 1;
        STACK[TOP] = element;
    }
}
```

```
void POP ( )
{
    int element ;
    if (TOP == -1)
    {
        printf("Stack is empty");
    }
    else
    {
        element = STACK [TOP];
        TOP = TOP - 1;
    }
}
```

```
printf("%d has been deleted", element);
}
}
void display ( )
{
    int i ;
```

```
if (TOP == -1)
    printf("STACK is empty");
else
{
    for (i=TOP; i>=0; i--)
    {
        printf("STACK %d", STACK[i]);
    }
}
```

```
void main ( )
{
    int choice ;
    do
    {
```

```
        printf("1 for Push /n 2 for Pop /n 3 for display /n 4 for exit");
        printf("Enter your choice");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1: Push ( ) ;
                    break;
            case 2: Pop ( ) ;
                    break;
            case 3: display ( ) ;
                    break;
            case 4: exit ( ) ;
                    break;
        }
    }
}
```

while (choice $\leq$ 4) ;

getch() ;

The main function calling by Operating System.

Application of STACK

(1) Polish and Reverse Polish Notation :

To simplification of any arithmetic expression using STACK is known as a Polish and Reverse Polish Notation.

There are three notations are used

(a) Prefix Notation

(b) Infix Notation

(c) Postfix Notation

- Polish Notation : In Polish Notation operator symbol are placed before its both operand is known as a Polish Notation. Prefix Notation is also known as Polish Notation.

Ex : +AB

- Reverse Polish Notation : In R.P.N. Operator symbol are placed after its both operand is known as a R.P.N. Postfix Notation is also known as a R.P.N.

Ex : AB+

Priority Level

- | | | |
|---------------|---|-----------------|
| ① Bracket () | → | 1 st |
| ② ↑ or ^ | → | 2 nd |
| ③ *, / | → | 3 rd |
| ④ +, - | → | 4 th |

Q. Convert infix expr. into postfix expression.

$$\Rightarrow A + B * C$$

$$\text{Sol} = A + (B * C)$$

$$\Rightarrow A + (BC*)$$

$$\text{Let } X = BC*$$

$$\Rightarrow A + X$$

$$\Rightarrow AX + \quad \Rightarrow ABC*+$$

Note:

Do not add any bracket in final answer.

Q. Infix into postfix expression: $A + B / C * D$

$$\Rightarrow A + (BC /) * D$$

$$\Rightarrow A + T * D$$

$$\Rightarrow A + (TD*)$$

$$\Rightarrow A + U \Rightarrow AU +$$

$$\Rightarrow ATD*+ \Rightarrow ABC/D*+$$

Q. Convert infix exp. into postfix exp.

$$(A+B) * C / D + E \uparrow F / G$$

$$\Rightarrow (AB+) * C / D + (E \uparrow F) / G$$

$$\Rightarrow T * C / D + U / G$$

$$\Rightarrow (TC*) / D + U / G$$

$$\Rightarrow 1D / + UG /$$

$$\Rightarrow 1D / UG / +$$

$$\Rightarrow TC * D / EF \uparrow G / +$$

$$\Rightarrow AB + C * D / EF \uparrow G / +$$

W.A algorithm to convert infix expression into postfix form using STACK.

Suppose 'Q' is an arithmetic expression written in infix form. This algorithm finds postfix expression 'P'.

- S-1: PUSH '(' onto the top of stack and add ')' to the end of Q.
- S-2: Scan Q from left to right & repeat steps 3 to 6 for each element of Q until the STACK is empty.
- S-3: If operand is encountered then add it to P.
- S-4: If left parenthesis is encountered then PUSH onto the top of STACK.
- S-5: If operator is encountered, then
- (a) PUSH onto the TOP of stack.
 - (b) Repeatedly POP from STACK & add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than coming operator.
- S-6: If right parenthesis is encountered:
- (a) Repeatedly POP from STACK and add to P each operator (on the top of stack) until the left parenthesis is encountered.
 - (b) Remove the left parenthesis.
- S-7: Exit.

Q.1 Convert infix exp. into Postfix form using STACK. The infix expression are given below:

$$Q: A + (B * C - (D / E \uparrow F) * G) * H$$

Q	STACK	P
A	(	A
+	(+	A
(	(+ (	A
B	(+ (	AB
*	(+ (*	AB
C	(+ (*	ABC
-	(+ (-	ABC *
(	(+ (- (	ABC *
D	(+ (- (	ABC * D
/	(+ (- (/	ABC * D
E	(+ (- (/	ABC * D E
$\uparrow$	(+ (- ($\uparrow$	ABC * D E
F	(+ (- (/ $\uparrow$	ABC * D E F
)	(+ (-	ABC * D E F $\uparrow$
*	(+ (- *	ABC * D E F $\uparrow$ /
G	(+ (- *	ABC * D E F $\uparrow$ / G
)	(+	ABC * D E F $\uparrow$ / G * -
*	(+ *	ABC * D E F $\uparrow$ / G * -
H	(+ *	ABC * D E F $\uparrow$ / G * - H
)	—	ABC * D E F $\uparrow$ / G * - H * +

Q.2 Convert infix Op. into Postfix form using STACK.
 $((A+B) * C)$

Q	STACK	P
(	(	
(	((	
A	((	A
+	((+	
B	((+	AB
)	((	AB +
*	((*	AB + C
C	((*	AB + C
)	((	AB + C
)	(	AB + C *

Q.3 - Convert infix exp. into postfix form using STACK.
 $A - B / C + D * E + F$

Q	STACK	P
A	(	A
-	(-	
B	(-	AB
/	(- /	AB
C	(- /	ABC
+	(+	ABC / -
D	(+	ABC / - D
*	(+ *	ABC / - D
E	(+ *	ABC / - D E
+	(+ +	ABC / - D E *
F	(+ +	ABC / - D E * F
)	(	ABC / - D E * F + +

Q. $A = B / (C * D + E)$

~~Q~~

	STACK	P
A	(	A
-	(-	A
B	(-	AB
/	(-/	AB
(	(-/ (	AB
C	(-/ (	ABC
+	(-/ ((+	ABC (/ -
D	(-/ ((+	ABC (/ - D)
↑	(+ ↑	ABC (/ - D
E	(+ ↑	ABC (/ - DE
)		
)		

A	(	A
-	(-	A
B	(-	AB
/	(-/	AB
(	(-/ (	AB
C	(-/ (	ABC
+	(-/ (*	ABC
D	(-/ (*	ABCD
↑	(-/ (* ↑	ABCD
E	(-/ (* ↑	ABCDE
)	(-/	ABCDE ↑ *
)	—	ABCDE ↑ * / -

Evaluation of Postfix Expression Using STACK

This algorithm find the value of arithmetic expression written postfix form P.

S-1: Add right paran. ')' to the end of P.

S-2: Scan P from left to right & repeat steps 3 & 4 for each element of P until the ')' is encounter.

S-3: If operand is encounter then PUSH onto the TOP of STACK.

S-4: If operator is encounter then,

(a) Remove two top elements from the STACK. Where A is the top element & B is next top element

(b) Evaluate $B \otimes A$.

(c) Place the result of 4(b) back onto the top of STACK.

S-5: Set value Equal to the TOP element of the STACK.

S-6: Exit.

Q. Evaluate Postfix expr. using stack the expr. are given b/w.

P = 5, 6, 2, +, *, 12, 4, /, -

5	Stack 5
6	5, 6
2	5, 6, 2
+	5, 9
*	40
12	40, 12
4	40, 12, 4
/	40, 3
-	37
)	37

Q. $P = 1, 2, +, 3, -, 2, 1, +, 3, \uparrow, +$

P	STACK
1	1
2	1, 2
+	3
3	3, 3
-	0
2	0, 2
1	0, 2, 1
+	0, 3
3	0, 3, 3
$\uparrow$	0, 27
+	29
)	29

Note: \$ and ** symbol is replaced by $\uparrow$ operator.

Queue

It is a linear type of data structure. It works on FIFO (First In First Out). Data may be insert at one end that end is known as a rear end. Data may be delete at other end that end is known as a front end.

Some conditions of Queue :

- * If insert the data element into the queue then $R = R + 1$
- * If delete element from queue then $Front = F + 1$
- * If only one element in the queue then $F = R$
- * If $R = \text{Max} - 1$ then overflow condition occur.
- * In simple queue front never be greater than Rear. If $F > R$ that means underflow condition.
- * If $F = -1$ and $F > R$ that indicate the underflow condition of queue.

* Write a program in C to implement Queue.

```
#include <stdio.h>
#include <conio.h>
#define Max 10
int Queue [Max];
int front = -1; Queue = -1;
```

void Insert - Queue ()

{

int element ;

if (R == Max-1)

{

printf("Queue is overflow");

}

else

{

if (F == -1)

F = 0;

printf("Enter the element");

scanf("%d", &element);

R = R + 1;

Queue[R] = element;

}

}

void Delete - Queue ()

{

int element ;

if (F == -1 || F > R)

{

printf("Queue is empty");

}

else

{

FRONT = FRONT + 1;

element = Queue [FRONT];

printf("%d has been deleted", element);

void display ()

{

int i;

if (F == -1 || F > R)

printf("Queue is empty");

else

{

for(i=FRONT; i<R; i++)

printf("%d", Queue[i]);

}

}

void main ()

{

int choice ;

do

{

printf("1 for Insert /n 2 for Delete /n 3 for Display /n 4 for exit");

printf("Enter your choice");

scanf("%d", &choice);

switch(choice)

{

case 1: Insert ();

break;

case 2: Delete ();

break;

case 3: Display ();

break;

(1)

(2)

(3)

(4)

(5)

(6)

(7)

(8)

(9)

(10)

```
case 4 : exit( );
```

```
while (choice <= 4);
```

* Write an Algorithm to Insert the data element into 'Queue'.

- (1) Create a 'Queue'.
- (2) Check the overflow condition. If $(Rear == max - 1)$ then print overflow condition occur.
- (3) Exit.
- (4) Check the value of front. If $(Front == -1)$ then $Front = 0$;
- (5) Insert the element into Queue.
 - (a) Set $Rear = Rear + 1$;
 - (b) Set $Queue[Rear] = Element$;
- (6) Exit.

* Write an Algorithm to Delete the data element.

- (1) Create a 'Queue'.
- (2) Check the underflow condition. If $(F == -1 || F > R)$ then print underflow condition occur.
- (3) Exit.
- (4) Delete the element from Queue.
 - (a) Set $Element = Queue[Front]$
 - (b) Set $F = F + 1$
- (5) Exit.

* Some Advantages of Circular Queue over Simple Queue *

- Circular Queue: The major drawback of simple queue where huge wastage of memory. This drawback is eliminated by circular queue. The explanation of Circular queue is given below:
- It is also linear type of Data Struct. It is also works on the FIFO Principle. This Queue is also used to different end like F & R

Some conditions of Circular Queue:

- (i) If insert the data element into circular queue then $R = (R+1) \% \text{Max Modulus Size}$
- (ii) If delete the element from circular queue then $F = (F+1) \% M \cdot S$
- (iii) If only one element in the circular queue then $F = R$
- (iv) If $F = (R+1) \% M \cdot S$ then overflow condition occur.
- (v) If $F = -1$ then, underflow condition occur
- (vi) $F > R$ is possible in C.Q. It doesn't show the underflow condition.
- (vii) $F = R$, it shows overflow.

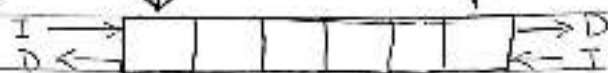
```
#include <stdio.h>
#include <conio.h>
#define Max 5
int Queue [Max]
; int F=-1, R=-1;
```

```
void Insert()
{
    int element;
    if (F == (R+1)%max)
        printf("Overflow");
    else
    {
        if (F == -1)
            F = 0;
        printf("Enter the element");
        scanf("%d", &element);
        R = (R+1)%max;
        Queue [R] = element;
    }
}
```

```
void Delete()
{
    int element;
    if (F == -1)
        printf("Underflow");
    else
    {
        if (F == R)
```

DE QUEUE

It stand for double-ended Queue. If In this type of Queue data may be insert or delete either both end of the Queue.



There are two types of DE Queue :

① I/P Restricted DE Queue :

In I/P Restricted DE Queue, restrict the input at the front end of the DE Queue.

② O/P Restricted DE Queue

Restrict the output at the rear end of the Queue.

[Imp] : Types of Operations Performed In DE Queue

- ① Insertion the element at the Rear End of DE Queue.
- ② Deletion of element at the Front End of DE Queue.
- ③ Insertion of element at the Front End of DE Queue.
- ④ Deletion of element at the Rear End of DE Queue.

∴ S-1 & S-2 are similar to Simple Queue.

∴ S-3 & S-4 $\Rightarrow$ If $(F=0)$ then overflow condition
Set element $F = F+1$

∴ S-4 $\rightarrow R = R-1$

$R=1$ then underflow condition.

① How many min. no. of queue required to implementing the priority queue? $\text{Min.} = 2$

Priority Queue

- Process : Process is defined as whose execution is started but not yet to be completed.

Process Control Block : Process can be represented in O.S by process control block. The main function of PCB is to maintain information of each process in O.S.

Linked List

<stdlib.h> - Header File

* Dynamic Memory Allocation :

- ① Malloc () $\Rightarrow$ Memory Allocation
- ② Calloc () $\Rightarrow$ Continuous Allocation
- ③ Realloc () $\Rightarrow$ Reallocation
- ④ Free ()

① Malloc () : void * malloc (size in byte) ; - Syntax

ptr = void * malloc (size in byte) ;

ptr = (cast-type*) malloc (10) ;

ptr = (int*) malloc (10) ; - Example

② Calloc () : void * calloc (no. of block, size of block in byte) ; - Syntax

ptr = void * calloc (no. of block, size of block in byte) ;

ptr = (cast-type*) calloc (3, 10) ;

ptr = (int*) calloc (3, 10) - Example

Difference b/w malloc & calloc :

- | | |
|--|--|
| ① Memory Allocation | ① Continuous Memory Allocation |
| ② It take one argument. | ② It take two argument. |
| ③ It makes variables without any initialization. | ③ Calloc gives with initialization with zero (0) ; |

(3) `Realloc()` : `void * Realloc (void * ptr, new size in byte) ;` - Syntax
Pointer `ptr` $\equiv$ `(void *) Realloc (ptr, new size in byte) ;`

LINKED LIST

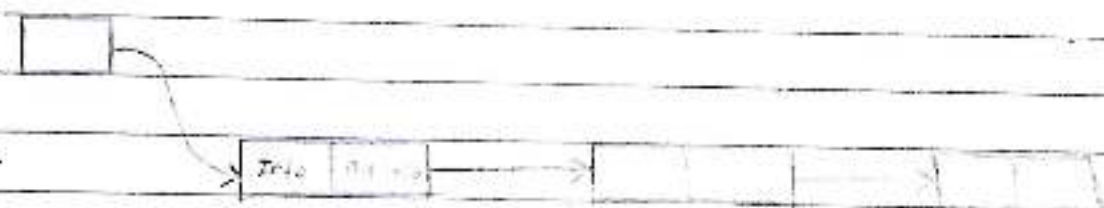
It is a linear collection of data element is called a Node. The linear order is given by means of pointer.

- * In linked list each Node are divided into two parts
- (1) Containing the information of any item &
 - (2) Containing the address of next node.

The second part of last node containing the null value that indicate the end of the list.

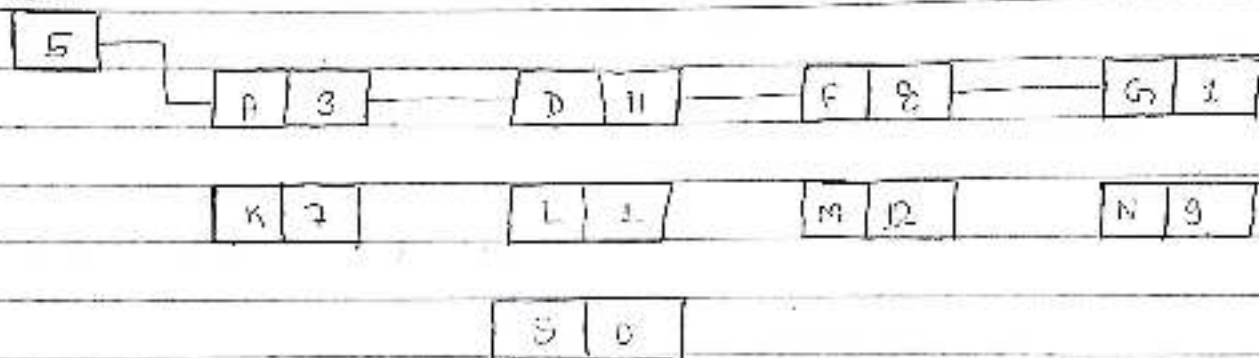
The start pointer hold the address of first node in the linked list.

START



	Info	Link
1	K	7
2		
3	D	11
4	M	12
5	A	3
6		
7	L	4
8	G	1
9	S	0
10		
11	F	8
12	N	9

START



Types of Linked List

- ① Singly Linked List
- (2) Circular Singly Linked List
- (3) Doubly Linked List
- (4) Circular Doubly Linked List

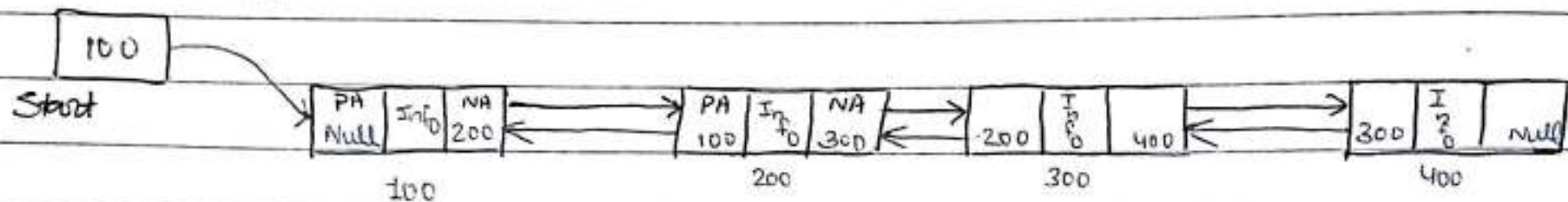
Types of operation in Linked List

- ① Insertion
- (2) Deletion
- (3) Traversing / Display

Doubly Linked List

In doubly linked list each node is divided into three parts :

- ① Containing address of previous node.
- ② Containing the information
- ③ Containing address of next node.



Third part of last node containing the null value that indicates the end of list & the first part containing the null value that indicates the this one would be

Start node of the linked list.

Advantage :

- ① It is highly efficient in respect to searching & traversing.

Disadvantage :

- ① Wastage of memory, because each address core store two time.

Circular Doubly linked List

struct dnode {

int data;

struct dnode *prev, *next;

};

Assignment

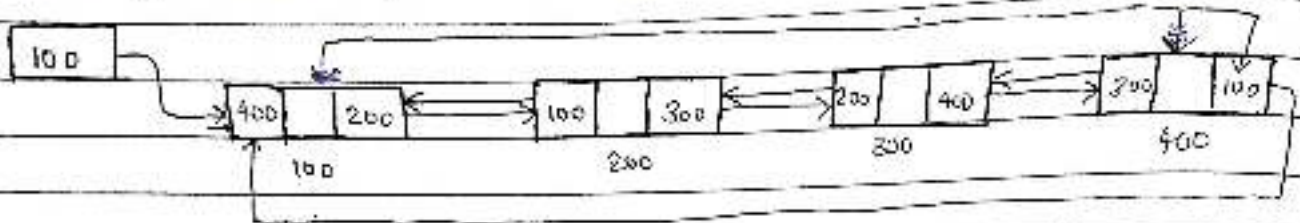
Circular Doubly Linked List

In circular doubly linked list each node are divided into three parts:

Containing address of previous node

Containing the information

Containing address of next node



Third part of last node containing the address of first node. &
the first part of first node contain the address of last node.
It indicates that it makes a circular linked list.

* Advantage:

It is highly efficient in respect to searching & traversing.

* Disadvantage:

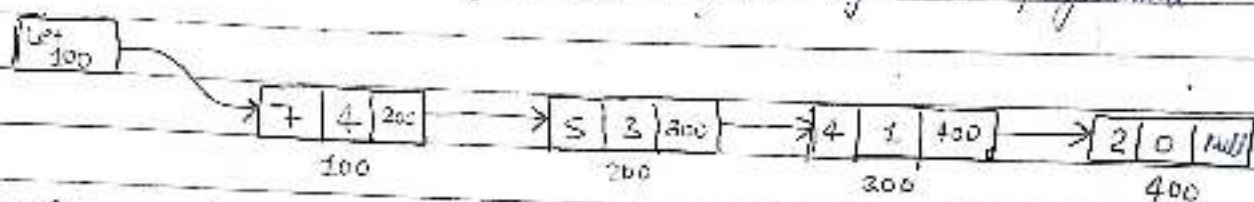
Wastage of memory because each address are store two time.

* Application of Linked List : Linked List are widely used to represent & manipulate the polynomial. Polynomials are containing non-zero coefficient & exponent. The following polynomials are given below :

$$x_1 = 7x^4 + 5x^3 + 4x + 2$$

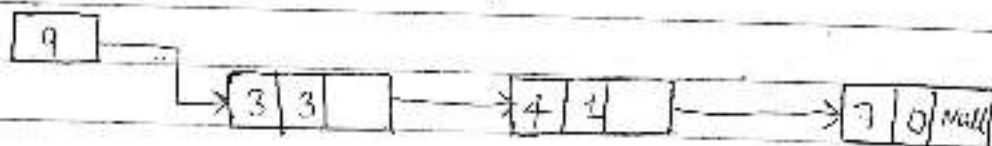
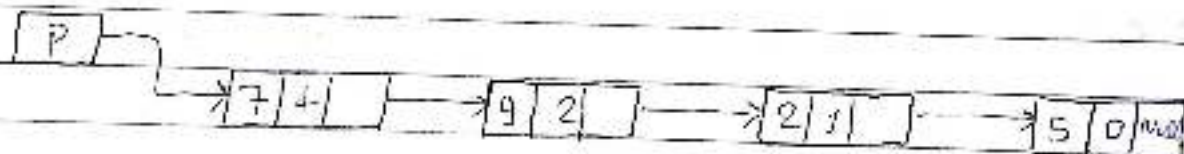
Linked List representation of polynomial each term of polynomial consider as a node. Each node are divided into three parts :

- ① Contains the coefficient value
- ② Contains the exponent value
- ③ Contains the address of next term of the polynomial.



Addition of two Polynomials Using Linked List

$$P(x) = 7x^4 + 9x^2 + 2x + 5 \quad \rightarrow \quad Q(x) = 3x^3 + 4x + 7$$



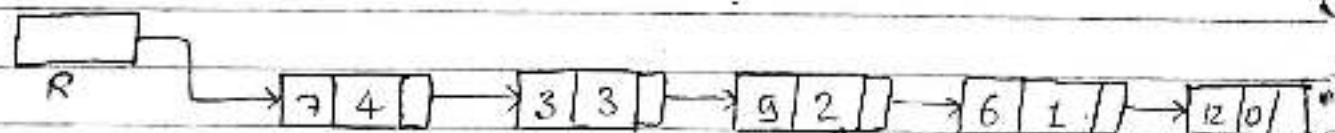
S-1 Comparing the exponent of P & Q. Here, exponent of P is greater than the exponent of Q. At the term pointed by P. And move the pointer P to their next node.

S-2: Compare the exponent of P & Q.

S-3: Compare the exponent of P & Q. Exponent of P is greater than exponent of Q. So, add the term pointed by P to the resultant linked list & move the pointer of P to their next node.

S-4: Again compare the exponent of P & Q. Both pointers having the equal position in term of exponent. So, add the coefficient value of both pointers & add to the resultant linked list. And move the pointer of P & Q to their next node.

S-5: Compare the exponent of P & Q. Both pointers having the equal position in term of exponent. So, add the coefficient of both pointers & add to resultant. And when moving the pointer to reach their null position & algorithm is terminated.



UNIT-4

TREE & GRAPH

Tree is a non-linear data structure. It is used to represent the hierarchical relationship existing among several data items.

- Tree is always non-cyclic.

Basic Terminology of Tree: There are different terminology used in tree:

- (1) Root
- (2) Node
- (3) Degree of Node
- (4) Degree of Tree
- (5) Terminal Node
- (6) Non-Terminal Node
- (7) Sibling
- (8) Level
- (9) Height / Depth
- (10) Edge
- (11) Path
- (12) Forest
- (13) Free face

• Root: The first element of the list are store in a node called root.

• Node: means containing the any type of information of any item.

• Degree of node: Each ^{particular} node has many children node are generated.

Ex - Degree of A = 3

• Degree of Tree: Whose node having max no. of children node generated in whole tree

• Terminal Node: That node which has no child node. (zero)

• Non-Terminal Node: That node which have atleast one child node.

• Sibling: Child node of a given parent.

• Level: Let level is denoted by 'n' & total level = $n+1$.

• Height/Depth: The level which is denoted by n is called depth.

• Edge: A line connecting two node.

• Path: In a given tree, the path from source to destination. We assuming that A is source & M is destination. The path from source to destination is $A \rightarrow D \rightarrow I \rightarrow M$.

• Forest: Remove the root node in given tree. The given tree will converted into the forest.

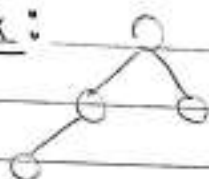
- Free tree : A tree which has no limit to generate child node.

Binary Tree

* Completed Binary Tree :

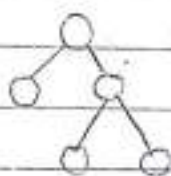
In complete binary tree filling of the data element from left to right & it hold the property of 2^n node at each level & the level before last level should be completed.

Ex :



* Strictly Binary Tree : Each node has either 0 children or 2 children.

Ex :



* Extended Binary Tree : Sometime known as a two-tree because two different nodes are used. A node with two child is called a internal node & represented by circle & denoted by N_i .

n tree = $n-1$ branch

A node with zero (0) children is called an external node & represented by rectangle & denoted by ' N_E '.

In extended binary tree external node N_E is always greater than to the internal node N_I . So, the relationship b/w these two.

$$N_E = N_I + 1$$

○ — Internal

□ — External

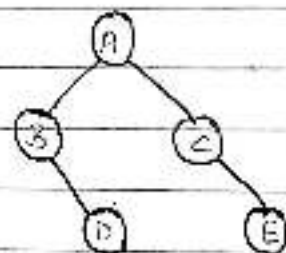
Representation of Binary Tree in Memory

There are two basic ways to represent a binary tree in memory:

- ① Static Memory Representation (Array Representation)
- ② Dynamic Memory Representation (Linked List Representation).

* Representation of Binary Tree in Array

- Array representation of binary tree, the root node element is stored at index 0. And left child of given binary tree is stored at index '1'. And right child is stored at index '2'. And this process will be repeated until the leaf level.



Size of Array = $2^{n+1} - 1 \Rightarrow 2^{2+1} - 1 = 2^3 - 1 = 7$

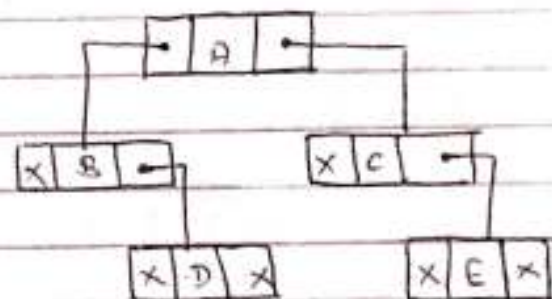
For given ex:

A	B	C	-	D	-	E
0	1	2	3	4	5	6

* Representation of binary tree in Linked List :

Each node are divided into three part

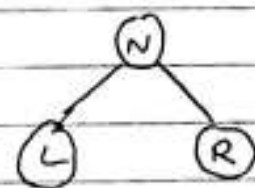
- ① Containing address of left child
- ② Containing the information
- ③ Containing address of right child.



BINARY TREE TRAVERSAL

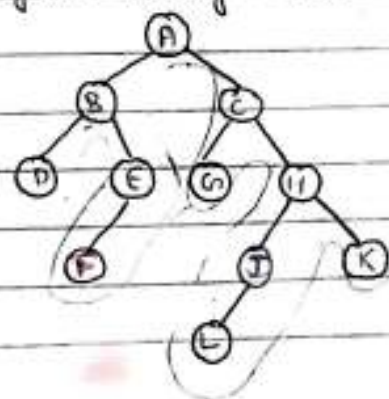
There are three methods to traverse a binary tree:

- ① Preorder Traversal - N L R
- ② Inorder Traversal - L N R
- ③ Post Order Traversal - L R N



Q.1 Find out the tree traversal sequences of a given binary tree

- Preorder : A B D E F C G H J L K
- Inorder : D B F E A G C L J H K
- Post order : D F E B G L J K H C A

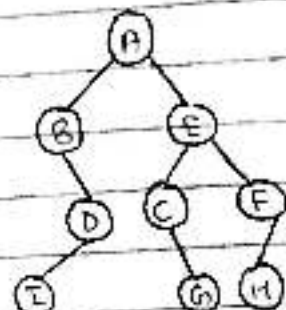


Q.2 Find out tree traversal sequence of a given binary tree.

Preorder: A B D I E C G F H

Inorder: B I D A G E H F

Postorder: I D B G C H F E A



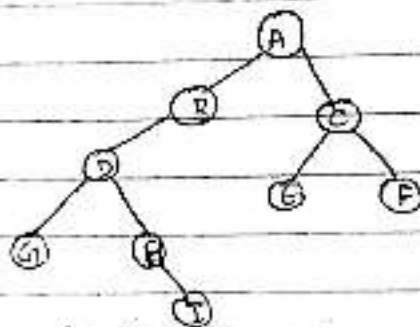
② Find the tree traversal sequences of a given binary tree.

A G I

Preorder: A B D G H I C E F

Inorder: G D H I B A E C F

Postorder: G I H D B E F C A

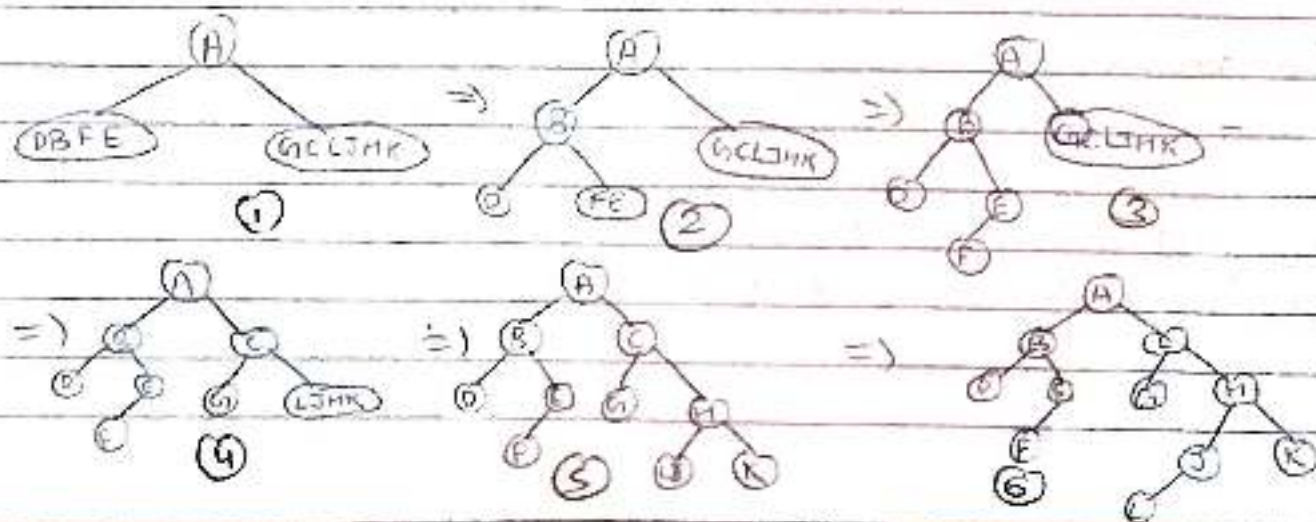


④ Construct a binary tree of a given tree traversal sequences.

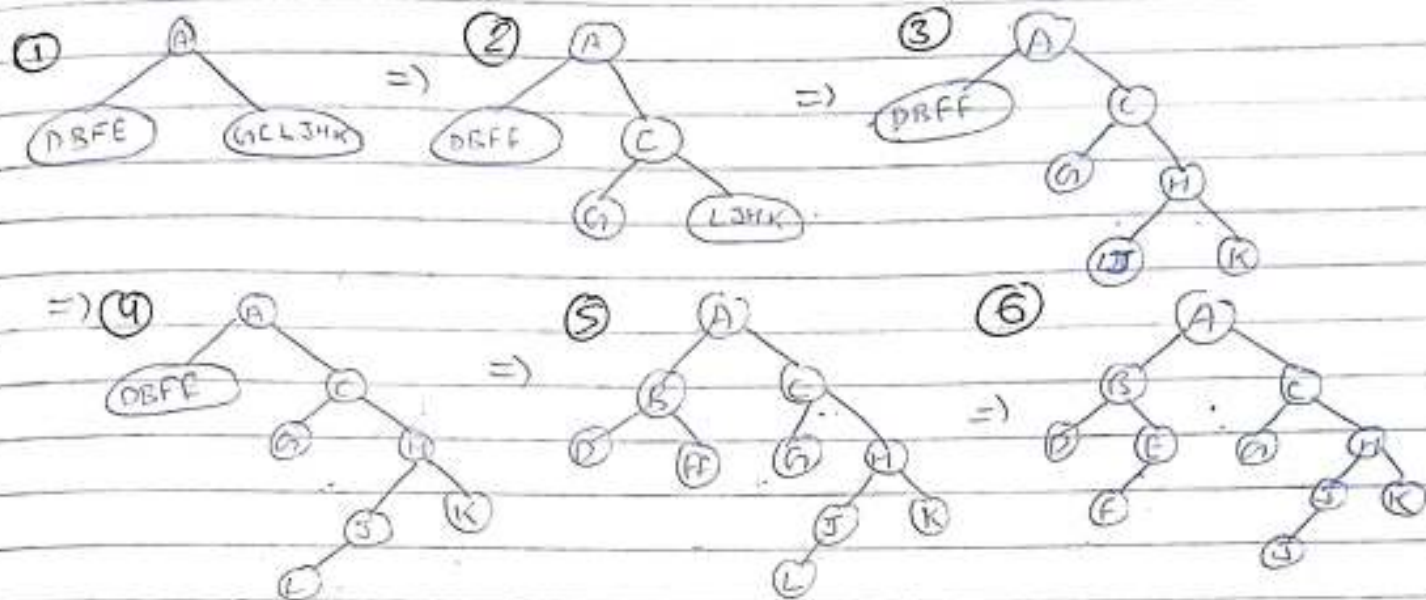
Preorder: A B D E F C G H J L K

Inorder: D B F E A G C L J H K

Postorder: D F E B G L J K H C A



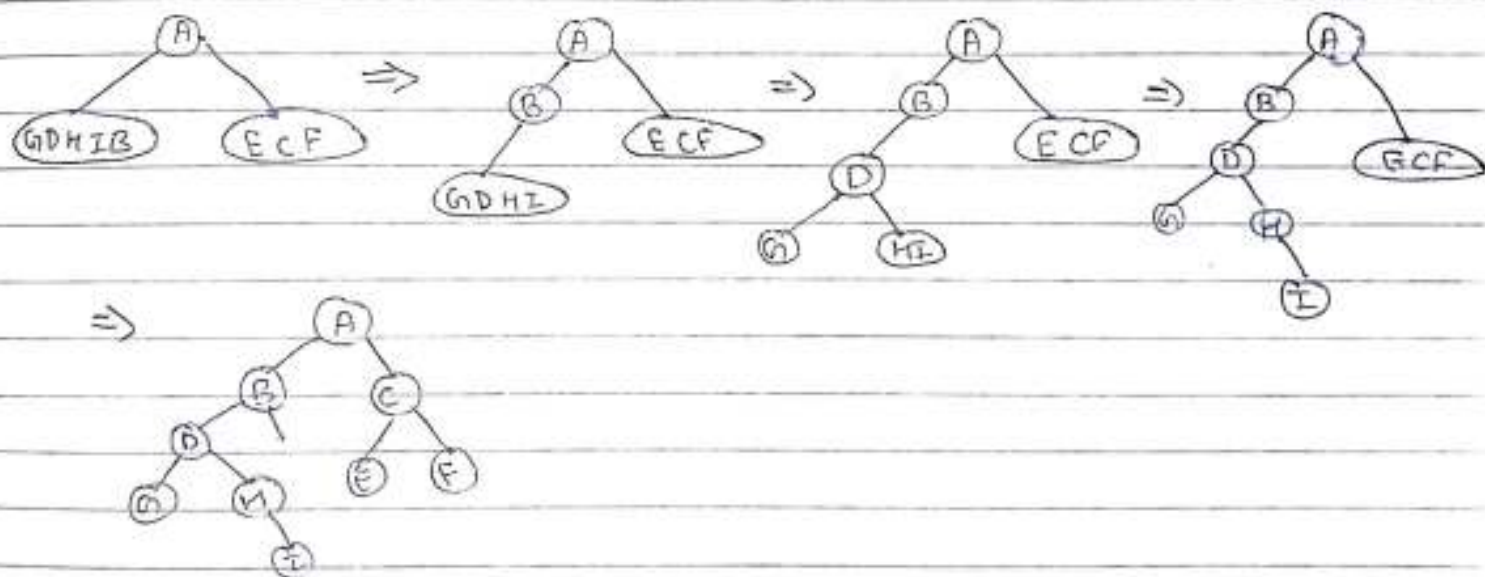
Inorder to Postorder



(5) Construct a binary tree of a given tree traversal sequences.

Preorder : A B D G H I C E F

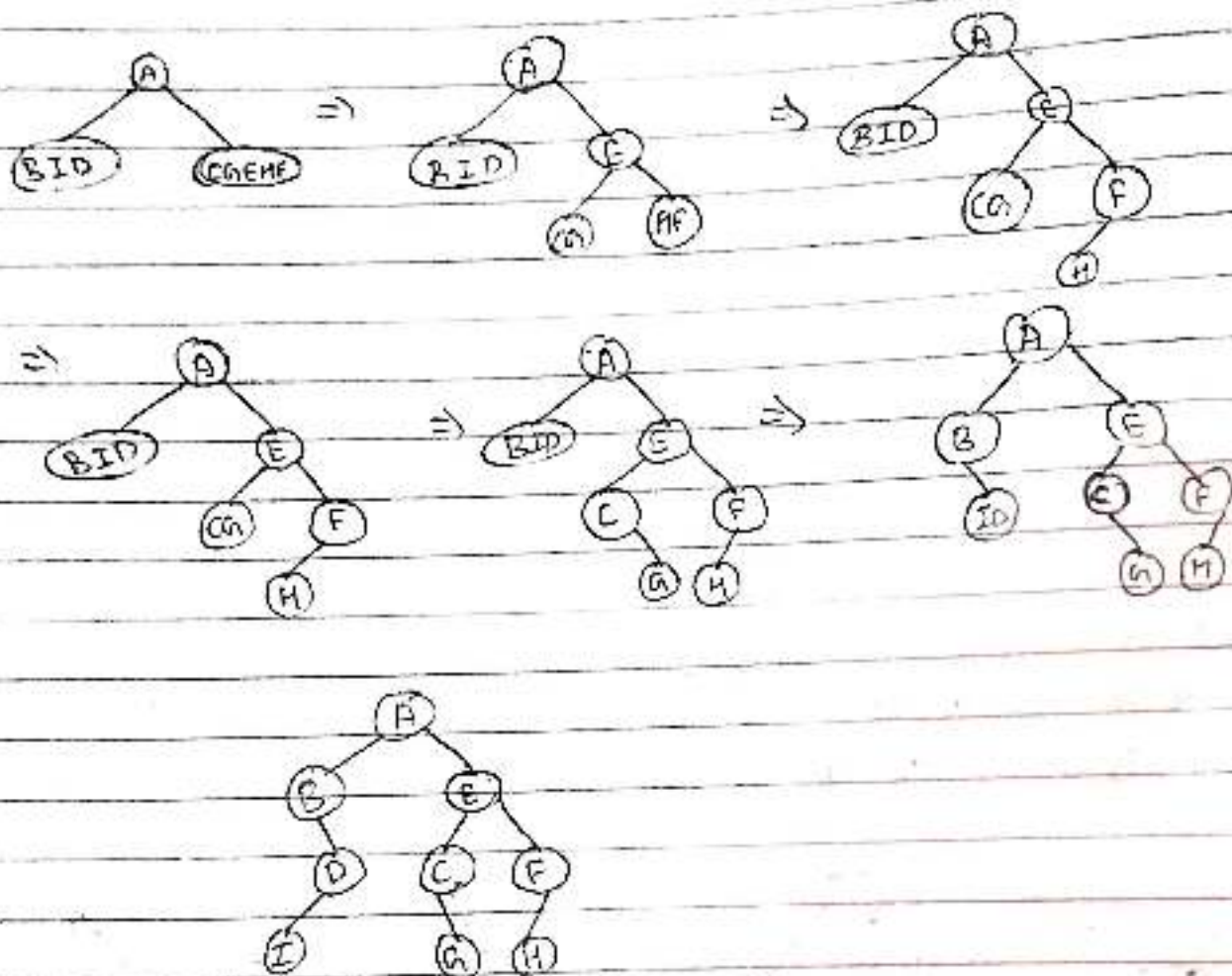
Inorder : G D H I B A E C F



⑤ Construct a binary tree :

Trailer: B I D A C G E H F

Post order : I D B G C H F E A



① Write an algorithm for Preorder tree traversal

- (1) Initially PUSH NULL on to STACK, and Initialize Ptr
Set $TOP = 0$, $STACK[0] = NULL$ and $Ptr = Root$
- (2) Repeat Steps 3 to 5 while $Ptr \neq NULL$
- (3) Apply process to INFO $[Ptr]$
- (4) [Right child?]
if $RIGHT [Ptr] \neq NULL$, then [PUSH on STACK]
Set $TOP = TOP + 1$ and $STACK [TOP] = RIGHT [Ptr]$
- (5) [Left child?]
if $LEFT [Ptr] \neq NULL$ then
Set $Ptr = LEFT [Ptr]$
else [POP from STACK]
Set $Ptr = STACK [TOP]$ and $TOP = TOP - 1$
- (6) Exit

(2) Write an algorithm for Inorder tree traversal.

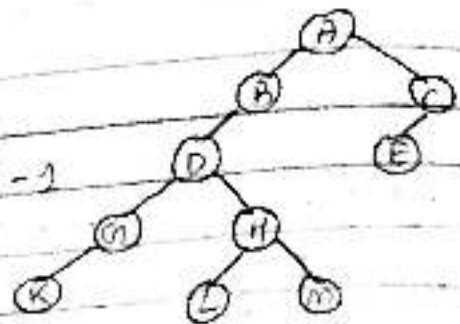
- (1) [PUSH NULL on to Stack and Initialize ptr]
Set $TOP = 0$, $Stack [0] = NULL$ and $ptr = Root$
- (2) Repeat while $ptr \neq NULL$ [Pushes left most path into Stack]
 - (a) Set $TOP = TOP + 1$
 - (b) Set $ptr = LEFT [ptr]$
- (3) Set $ptr = Stack [TOP]$ and $TOP = TOP - 1$
- (4) Repeat steps 5 to 7 while $ptr \neq NULL$
- (5) Apply process to INFO $[ptr]$
- (6) [RIGHT CHILD?]
if $RIGHT [ptr] \neq NULL$ then,

(a) Set $ptr = RIGHT[ptr]$

(b) Go to Step 2

7 Set $ptr = STACK[TOP]$ and $TOP = TOP - 1$

8 Exit.



BINARY SEARCH TREE (B.S.T)

If smaller element than the root node are inserted to the left part of the tree & if elements are larger than the root node then they are inserted to the right part of the tree
($O(\log_2 n)$)

Deletion in Binary search tree:

There are three cases generated if we are deleting the element / node from a given binary search tree.

Case 1: If node having zero child, there is no effect on the binary search tree after deleting that node.

Case 2: A node having one child the deleted node replaced by immediate predecessor ~~or~~ immediate successor of that node. That deleted node is belongs to level before last level not a intermediate node.

Case 3: A node having two child that node are replaced by immediate successor of in order value of a given binary tree. If immediate successor is not found then replaced by immediate predecessor.

AVL Tree (Height Balance Tree)

The AVL tree are sometimes known as Height Balance Tree. This tree invented by those Russian Mathematician

Adelson, Velsiki, Landis.

So, we can call this tree is a AVL Tree.

- Each node of an AVL tree has the property the height of the left sub tree is $+1, 0, -1$ to the height of the right sub tree. It also follows the binary search tree property.

$$\text{So, } \boxed{\text{Balance Factor} = \text{Height of L.S.T} - \text{Height of R.S.T}}$$

Types of Rotation in AVL Tree

There are four types of Rotation:

- ① Left - Left Rotation
- ② Left - Right Rotation
- ③ Right - Left Rotation
- ④ Right - Right Rotation

Note: ① If unbalanced node & immediate child of that node both are having a positive sign that indicates the single rotation problem & rotate the unbalanced node to the clockwise direction.

② If unbalanced node & immediate child of node ^{both} having the negative sign then single rotations required the unbalanced node are rotate to the anticlockwise direction.

③ If unbalanced node & immediate child of that node having a opposite sign. So, this problem is belong to double rotation problem where both nodes are rotate at once. According to Note-1 & 2

Q.1 : Construct an AVL Tree of a given data elements.
B F I N L P J A H D

* GRAPH *

Graph is a non-linear data structure it is a finite set of vertices (V) & edge (E). It is used to represent the network model existing several data items. There is no unique node (vertex) V. Graph may be cyclic or non-cyclic. So.

$$G = \{V, E\}$$

Difference b/w Tree & Graph

TREE

- (1) Non-linear data structure.
- (2) It is a collection of node & edge. So.

$$T = \{N, E\}$$

- (3) Non-cyclic always.
- (4) Used to represent the hierarchical model.
- (5) It has one unique node i.e. Root node.

GRAPH

- (1) Non-linear data structure.
- (2) It is a collection of vertex & edge.

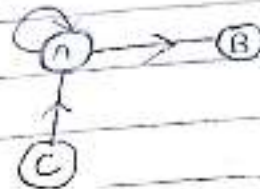
$$G = \{V, E\}$$

- (3) Graph may be cyclic or non-cyclic.
- (4) Used network model.

- (5) It doesn't have any unique node/vertex.

Basic Terminology of Graph

(1) Directed Graph :



Set of vertex $(V) = (A, B, C)$

Set of edge $(E) = (A, A), (A, B), (C, A)$

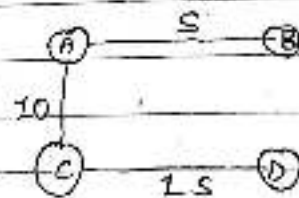
(2) Undirected Graph :

Set of vertex $(V) = (A, B, C)$

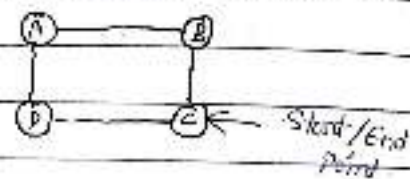
Set of Edge $(E) = (A, A), (A, B), (A, C), (C, A), (B, D)$

(3) Weighted Graph : It is a graph in which edges are assigned with non-negative weight.

It may be directed or undirected.



(4) Cyclic Graph : It is a simple path where start & end at the same vertex.



It also may be directed or undirected.

These are two popular algorithm to traverse a graph

Depth First Search (DFS) $\Rightarrow$ Stack

Breadth First Search (BFS) $\Rightarrow$ Queue

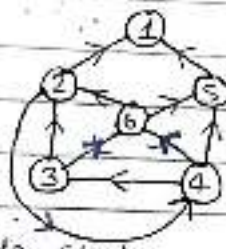
Q. Traverse a given graph with the help of DFS.

Stack Vertex Vertex Visited Action

Stack: [] Vertex: 1 2 3 4 5 6
[0][0][0][0][0][0]

—

Initialization



Stack: [1] Vertex: 1 2 3 4 5 6
[0][0][0][0][0][0]

—

Push initial vertex into stack

Stack: [2] Vertex: 1 2 3 4 5 6
[1][0][0][0][0][0]

1

POP(1), Visit(1) & Push adjacent vertex of 1 into stack.

Stack: [3] Vertex: 1 2 3 4 5 6
[1][1][0][0][0][0]

1, 2

POP(2), Visit(2) & Push adjacent vertex of 2 into stack

Stack: [3, 4] Vertex: 1 2 3 4 5 6
[1][1][1][0][0][0]

1, 2, 4

POP(4), Visit(4) & Push all adjac. vertex of 4 into stack.

Stack: [3, 5, 6] Vertex: 1 2 3 4 5 6
[1][1][1][1][0][0]

1, 2, 4, 6

POP(6), Visit(6) & Push all adjac. vertex of 6 into stack.

Stack: [3, 5] Vertex: 1 2 3 4 5 6
[1][1][1][1][1][0]

1, 2, 4, 6, 5

POP(5), Visit(5) & Push all adjac. vertex of 5 into stack

Stack: [3] Vertex: 1 2 3 4 5 6
[1][1][1][1][1][1]

1, 2, 4, 6, 5, 3

POP(3)

Stack: [] Vertex: 1 2 3 4 5 6
[1][1][1][1][1][1]

1, 2, 4, 6, 5, 3

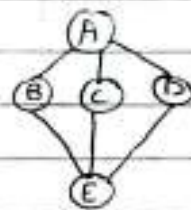
POP(3), Visit(3) & Graph is terminated

Q. Traverse a given graph with the help of BFS.

Queue	Vertex	Vertex Visited	Action
	1 2 3 4 5 6 0 0 0 0 0 0	—	Initial
1	1 2 3 4 5 6 0 0 0 0 0 0	—	Insert initial vertex in queue
2	1 2 3 4 5 6 1 0 0 0 0 0	1	Delete(1), Visit(1) & Insert adjac. vertex into queue
4	1 2 3 4 5 6 1 1 0 0 0 0	1, 2	Delete(2), Visit(2) & Insert adjac. vertex into queue
3 5 6	1 2 3 4 5 6 1 1 0 1 0 0	1, 2, 4	Delete(4), Visit(4) & Insert adjac. vertex into queue
5 6	1 2 3 4 5 6 1 1 1 1 0 0	1, 2, 4, 3	Delete(3), Visit(3) & Insert adjac. vertex into queue
6	1 2 3 4 5 6 1 1 1 1 1 0	1, 2, 4, 3, 5	Delete(5), Visit(5)
6	1 2 3 4 5 6 1 1 1 1 1 1	1, 2, 4, 3, 5, 6	Delete(6), Visit(6)
	1 2 3 4 5 6 1 1 1 1 1 1	1, 2, 4, 3, 5, 6	Delete(6)

ASSIGNMENT - 2

Traverse a given graph with the help of DFS & BFS.



Stack	Vertex	Vertex Visited	Action
	A B C D E 0 0 0 0 0	—	Initial
A	A B C D E 1 0 0 0 0	A	Insert initial vertex
B C D	A B C D E 1 1 0 0 0	A	POP(A), Visit(A), Push adjacent vertices
B C E	A B C D E 1 1 0 1 0	A, D	POP(D), Visit(D), PUSH adjacent vertex
B C B C	A B C D E 1 1 0 1 1	A, D, E	POP(E), Visit(E), PUSH adjacent vertex
B C B	A B C D E 1 1 1 1 1	A, D, E, C	POP(C), Visit(C), PUSH adjacent vertex
A C	A B C D E 1 1 1 1 1	A, D, E, C, B	POP(B), Visit(B), PUSH adjacent vertex
B	A B C D E 1 1 1 1 1	A, D, E, C, B	POP(C)
	A B C D E 1 1 1 1 1	A, D, E, C, B	POP(B)

Queue

Vertex

Vertex Visited

Action

[] [] [] [] []

A	B	C	D	E
0	0	0	0	0

-

Initial

[A] [] [] [] []

A	B	C	D	E
0	0	0	0	0

-

Insert initial vertex

[B] [C] [D] [] []

A	B	C	D	E
1	0	0	0	0

A

Delete(A), Visit(A) & ^{Insert} ~~Insert~~ adjacent vertex

[C] [D] [E] [] []

A	B	C	D	E
1	1	0	0	0

A, B

Delete(B), Visit(B) & ^{Insert} ~~Insert~~ adj. vertex

[D] [E] [E] [] []

A	B	C	D	E
1	1	1	0	0

A, B, C

Delete(C), Visit(C) & ^{Insert} ~~Insert~~ adj. vertex

[D] [E] [D] [] []

A	B	C	D	E
1	1	1	1	0

A, B, C, E

Delete(E), Visit(E) & ^{Insert} ~~Insert~~ adj. vertex

[D] [E] [] [] []

A	B	C	D	E
1	1	1	1	1

A, B, C, E, D

Delete(D), Visit(D) & ^{Insert} ~~Insert~~ adj. vertex

[D] [] [] [] []

A	B	C	D	E
1	1	1	1	1

A, B, C, E, D

Delete(E)

[] [] [] [] []

A	B	C	D	E
1	1	1	1	1

A, B, C, E, D

Delete(D)

Minimum Spanning Tree (MST)

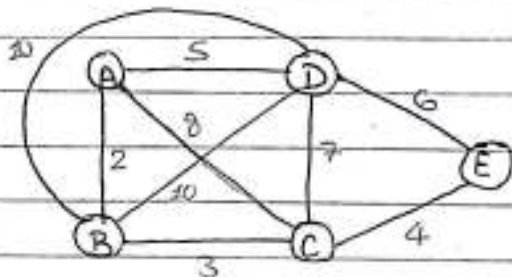
It must be weighted & undirected. It is always be cyclic

Spanning tree is a application of weighted graph. We remove the some edges along with maximum weight that given graph must be cyclic and always undirected graph. After the reconstruction of graph. Graph must be non-cyclic. There are two popular algorithm to construct the spanning tree

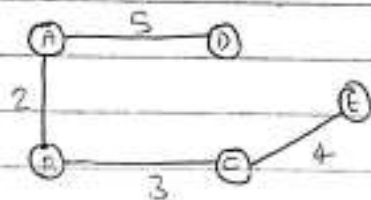
- (1) Kruskal's Algorithm + (it works on Edges)
- (2) Prim's Algorithm + (it works on Vertex)

Q.1 Construct the spanning tree & also find out the MST of a given graph using Kruskal's algorithm.

Edges are sorted in ascending order of their weights



- | | |
|---------|---|
| AB = 2 | ✓ |
| BC = 3 | ✓ |
| CE = 4 | ✓ |
| AD = 5 | ✓ |
| DE = 6 | x |
| DC = 7 | x |
| AC = 8 | x |
| BC = 10 | x |



Spanning tree, Minimum cost of spanning tree
= $2 + 3 + 4 + 5 = 14$

Q. Construct spanning tree of a given graph using Kruskal's algorithm. Also, find MCST.

$$DF = DE = 1$$

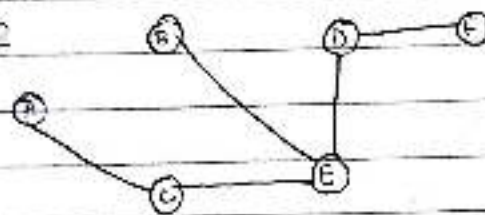
$$BE = AC = 2$$

$$x BD = CE = 3$$

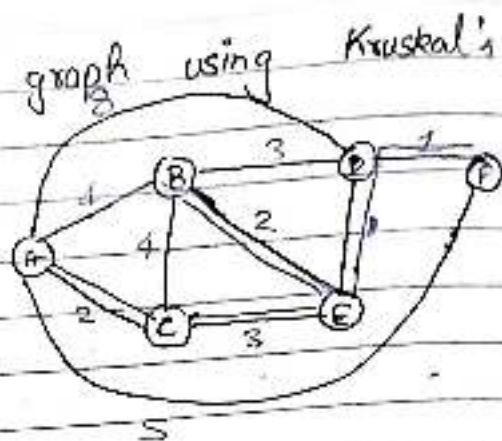
$$x BC = BA = 4$$

$$x AF = 5$$

$$x AD = 8$$



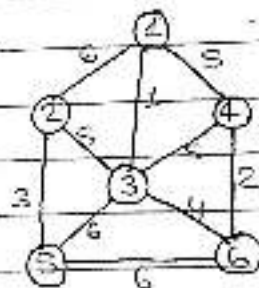
Spanning tree



$$\therefore 1+1+2+2+3 = 9 \Rightarrow \text{MCST} = 9$$

Q.3 Find out MCST of a given graph using Prim's algorithm.

Note: If initial vertex is not given in the problem. So, we can assume that the smallest no. of vertex as a initial.



$$\text{Vertex 1} = \frac{1-2}{6}, \frac{1-3}{1}, \frac{1-4}{5}$$

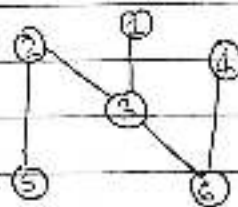
$$\text{Vertex 3} = \frac{3-2}{5}, \frac{3-4}{4}, \frac{3-5}{6}, \frac{3-6}{4}$$

$$\text{Vertex 6} = \frac{6-4}{2}, \frac{6-5}{6}$$

$$\text{Vertex 4} = -$$

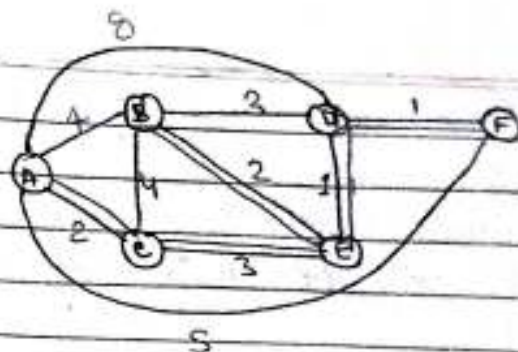
$$\text{Vertex 2} = \frac{2-3}{5}$$

$$\text{Vertex 5} = -$$



$$\text{Total Cost of S.T} = 1+4+2+3+5 = 15$$

(2) Find out MCST of a given graph using Prim's algorithm.
 x -initial = F



$$\therefore FD = 1 \checkmark$$

$$FA = 5$$

$$\Rightarrow DB = 3 \checkmark$$

$$DE = 1 \checkmark$$

$$\Rightarrow FB = 2 \checkmark$$

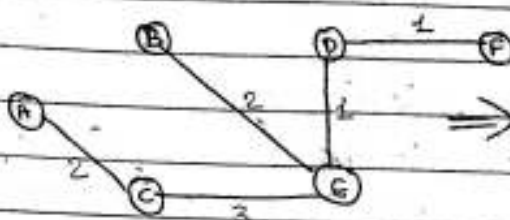
$$EC = 3 \checkmark$$

$$\Rightarrow BC = 4$$

$$BA = 4$$

$$\Rightarrow CA = 2 \checkmark$$

$$\Rightarrow AD = 8$$



$\Rightarrow$ Spanning Tree

$$\text{Min. cost spanning tree} = 1 + 1 + 2 + 3 + 2$$

$$= 9$$

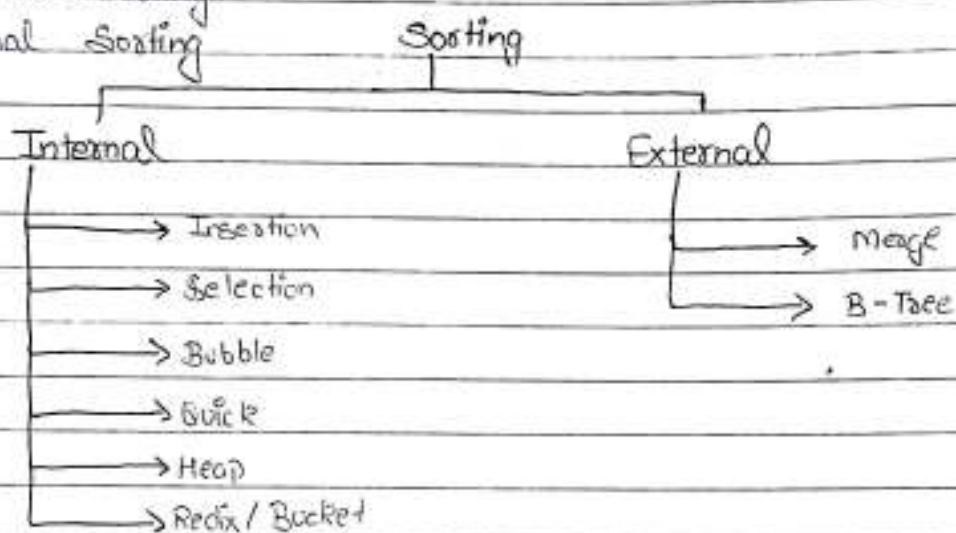
Sorting

In Sorting data elements are arranged in increasing or decreasing order on the basis of users requirement.

There are two types of Sorting.

① Internal Sorting

② External Sorting



(1) Works only on small data, data reside in the main memory.

(2) Requires only sorting process.

(3) Faster

Ex: Quick, Insertion, Heap

(1) Data list is large enough not to fit in main memory.

(2) Requires sorting & merging process.

(3) Slower because of involvement of Secondary Storage device.

Ex: Merge, B-Tree

Quick Sort

It is one of the best sorting technique, it is work on divide or conquer method. One key element is very impor. In quick sort the Pivot element.

Pivot element means the first element of the list is known as a Pivot element.

What are the major role play in Quick Sort Technique?

Algorithm of Quick Sort

(1) Set $LEFT = BEGIN$, $RIGHT = END$, and $LOC = BEGIN$

(2) [Scan from $RIGHT$ to $LEFT$]

(a) Repeat while $A[LOC] \leq A[RIGHT]$ and $LOC \neq RIGHT$
 $RIGHT = RIGHT - 1$

(b) if $LOC = RIGHT$ then, Return

(c) if $A[LOC] > A[RIGHT]$ then

(i) Interchange $A[LOC]$ and $A[RIGHT]$

$TEMP = A[LOC]$, $A[LOC] = A[RIGHT]$

$A[RIGHT] = TEMP$

(ii) Set $LOC = RIGHT$

(iii) Go to Step 3

(3) [Scan from LEFT to Right] to

(a) Repeat while $A[LEFT] \leq A[LOC]$ and $LEFT \neq LOC$

$LEFT = LEFT + 1$

(b) if $LOC = LEFT$, then return

(c) if $A[LEFT] > A[LOC]$ then

(i) Interchange $A[LEFT]$ and $A[LOC]$

$TEMP = A[LOC]$, $A[LOC] = A[LEFT]$

$A[LEFT] = TEMP$

(ii) Set $LOC = LEFT$

(iii) Go to step 2.

(4) Exit.

Q. W.A.A for quick sort & also trace the algorithm for given data element : $(44), 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, (66)$

$\Rightarrow$ $(22), 33, 11, 55, 77, 90, 40, 60, 99, (44), 88, 66$

R-F

$\Rightarrow$ $22, 33, 11, (44), 77, 90, 40, 60, 99, (55), 88, 66$

L-R

$\Rightarrow$ $22, 33, 11, (40), 77, 90, (44), 60, 99, 55, 88, 66$

R-L

$\Rightarrow$ $22, 33, 11, 40, (44), 90, (77), 60, 99, 55, 88, 66$

L-R

R-F

Heap Sorting

It is almost complete binary tree the filling of the data elements from left to right. There are two types of heap:

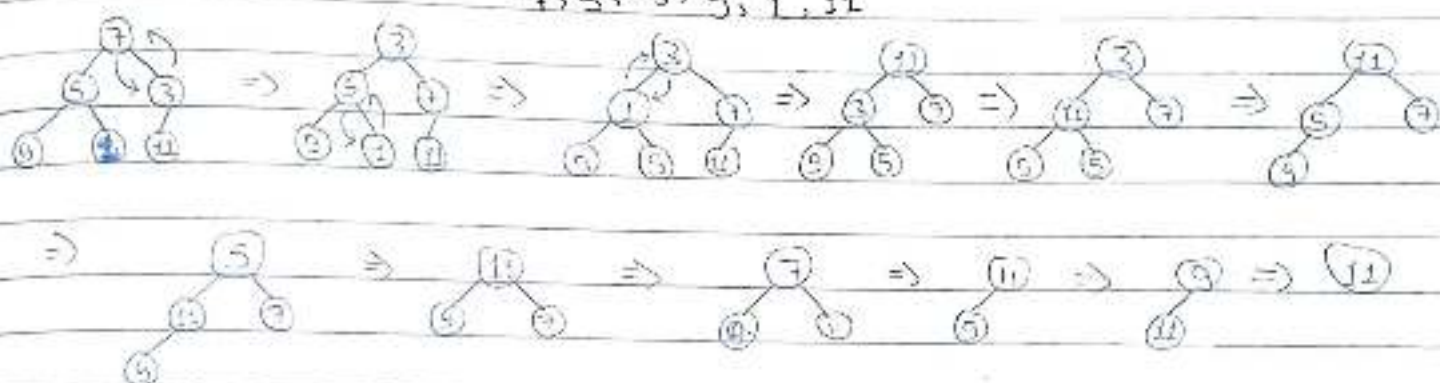
- ① Min Heap
- ② Max Heap

In min heap parent node is always smaller than its children & in max heap parent node is always larger than its children

Min heap example :

0	1	2	3	4	5
1	3	5	7	9	11

7, 5, 3, 9, 1, 11



Searching

In searching, find the location of any elements are stored in array. There are two searching technique.

- (1) Linear search / Sequential Search
- (2) Binary Search

Linear Search

- ① Linear search technique is work on sorted or unsorted array.
- ② Linear search searching the element in sequential manner.
- ③ Searching time complexity of linear search is order of n .
- ④ Linear is less efficient.
- ⑤ $i = 0, i = i - 1$

5	10	2	9	13	15
0	1	2	3	4	5

Binary Search

- ① Binary search technique is work on only sorted array.
- ② Binary search searching the element in random manner.
- ③ Searching time complexity of binary search is order of $\log_2(n)$.
- ④ Binary is more efficient.
- ⑤

5	10	2	9	13	15	18	20
0	1	2	3	4	5	6	7

$$\text{Mid}[n] = \frac{0+7}{2} = 3.5$$

$\Rightarrow$ Applying ceiling, $\lceil 3.5 \rceil = 4 = \text{Mid}[n]$

$$\therefore \text{New LB} = \text{Mid}[n] + 1 \\ = 4 + 1 = 5$$

$$\therefore \text{Mid}[n] = \frac{5+7}{2} = 6$$

Hashing or Hash Addressing

Searching time of linear & binary search totally dependent on the no. of n element. If no. of n element increases then searching time must be increased.

Hashing is another searching technique which is essentially independent of the no. of n element.

$H: K \rightarrow L$, where, H is Hash function,

K is set of Keys

L is set of memory addresses

This topic hashing divided into two major part

(1) Hash function

(2) Collision Resolution Strategy

* Hash Function: There are two principle criteria one use in selecting a Hash function.

(a) Easy & quick to compute

(b) Minimum no. of collision occur.

There are different type of Hash Function:

(1) Division Method: Select a no. ' m ' larger than the no. ' n ' of key ' K '. The m is a prime no. This Hash function H is defined as $H(K) = K \bmod (m)$

(2) Mid Square Method: The key K is squared then H is defined as $H(K) = \lfloor \frac{K^2}{p} \rfloor$

where p is obtained by deleting digits from both end of K^2 .

(3) Folding method: The key K is partition into a no. of parts $K_1, K_2, \dots, K_n$, where each part ~~are~~ except the last, has the same no. of digits as the required address, then parts are added together and ignoring the last carry.

$$H(K) = K_1 + K_2 + K_3 + \dots + K_n$$

Q. Consider a company where each Employee is assigned a unique 4 digit employee no. The total no. of employee in a company is 99. Suppose L consist of 100, 2 digit address. Apply the all Hash function & find out location of given key elements: (i) 3205 (ii) 7148 (iii) 2345

Division

Mid Square

Folding

$$\Rightarrow H(3205) = 3205 \% 99 \\ = 05$$

$$K^2 = 10272025 \\ H(K) = 99$$

$$\Rightarrow 3205 = 32 + 05 \\ H(K) = 37$$

$$H(7148) = 7148 \% 99 \\ = 67$$

$$K^2 = 51093904 \\ H(K) = 93$$

$$\Rightarrow 7148 = 71 + 48 = 119 \\ H(K) = 19$$

$$H(2345) = 2345 \% 99 \\ = 17$$

$$K^2 = 5499025 \\ H(K) = 99$$

$$\Rightarrow 2345 = 23 + 45 = 68 \\ H(K) = 68$$

• Truncate:
 $H(3205) = 05$
 $H(7148) = 48$
 $H(2345) = 45$

Collision Resolution

Collision Resolution
Suppose we want to add a new record 'R' with key 'K' in file F. But the memory location address H(K) is already occupied. So, this situation is known as a collision.

These are different collision resolution strategy:
(Extend Hashing)

(Cloned Harking)

(1) Open addressing : linear Probing & modification, Quadratic Probing

Suppose that a new record 'R' with key 'K' is to be added to the memory table 'T' but that the memory location with hash address $H(K)$ is already filled. This technique to resolve the collision is to assign or to be first available location.

(2) Chaining Method (Open Hashing) :

- Load Factor : $\lambda = \frac{n}{m}$ (Total no. of key value / no. of slots address)

- Probes: Key comparisons

MCA

Algorithm for Insertion

Insert (A, K, N, TCM)

Linked List:

```
typedef struct node {  
    int data;  
    struct node* next;  
} node;
```

• Linear Probing Example:

Consider some elements & take table size to be 11. 29, 18, 43, 10, 36, 25, 46

• Quad-Probing Example: 29, 18, 43, 10, 46, 54

00	10
01	
02	46
03	54
04	
05	
06	
07	29
08	18
09	
10	43

$$H(29) = 29 \% 11 = 07$$

$$H(18) = 18 \% 11 = 07$$

$$43 \% 11 = 10$$

$$10 \% 11 = 10$$

$$46 \% 11 = 02$$

$$54 \% 11 = 10$$

• Double Hashing: $H(k) = k \% 11$ — Example

$$H'(k) = 11 - (k \% 11)$$

$$\therefore H + H' = 11 \quad \text{--- (1)}$$

$$\therefore H + 2H' = 11 \quad \text{--- (2)}$$